Characterizing the Cornerstone Octree Construction Method

Henry Murdock, Author Name, Rayan Singh
University of Illinois at Urbana-Champaign
{henrygm3,netid2,rayans2}@illinois.edu

Abstract

An octree is a structure which partitions a 3D space into octants recursively based on positions of objects within the domain. This structure is heavily utilized in scientific simulation, computer graphics, and collision detection. A classical use case of an octree is in the field of particle simulations such as those involving short or long range forces where the number of particles required is large enough to become prohibitive for the simulation.

Recent works have expanded upon traditional octree construction methods to allow for distributed simulation termed Locally Essential Trees (LETs) as well as how to effectively parallelize their construction. We explore and benchmark a novel method called Cornerstone that not only utilizes these methods but develops a construction method that allows an octree to be rebalanced which is favorable for particle simulations or any problem which expresses perturbations of the positions of represented objects.

1 Introduction

Scientific simulation, computer graphics, and collision detection as well as other fields utilize structures called octrees. An octree allows a three dimensional space to be broken down hierarchically into a tree structure with up to eight children per node. Meshless point clouds, such as a group of particles, are an important application of such structures, especially when paired with Barnes-Hut or Fast Multipole Methods which allow reductions in the runtime of force calculations between points from $O(n^2)$ to $O(nlog(n))$ and roughly $O(n)$ respectively. This has driven the development of octree construction methods that operate on GPUs and across multiple nodes in clusters and supercomputers.

One of the most interesting new methods for meshless point clouds is Cornerstone [1]. In the case of simulating the movement of particles in dynamic systems, the octree must be recalculated at each time step, however the primary advantage of Cornerstone is to rebalance the tree rather than rebuild from scratch. Cornerstone has been heavily used on exascale supercomputers, however not all problems which may benefit from this method require such systems. Furthermore a profile of the code internals and how the method performs across varying conditions of particle distributions, parameters, and other variants does not appear publicly but may help to optimize and evidence the methods usefulness across varying systems and problem spaces. This project seeks to do such a characterization and will focus closely on the mapping of Cornerstone to the current GPU architectures examining any kernel level metrics of interest, memory movement at the intra and inter node scope, as well as the interactions of communication and computation cost with different algorithm parameters.

2 Cornerstone

To characterize Cornerstone's performance we first need to understand the algorithmic steps. This involves iterating over several elementwise functions until reaching a convergence criterion.

2.1 Space Filling Curves

The first step which is common to any octree construction method is the calculation of the space filling curve. Informally a space filling curve will take points existing in space and map them to integer keys, when one sorts the original particles based on these keys and traverses the path through them the resulting curve will recursively move through the domain octant by octant (in the case of three dimensional points). In this way a space filling curve is a kind of fractal.

We are interested in two types of space filling curves namely Morton Z-order curves and Hilbert curves. The path of a Morton Z-order curve may be viewed in Figure 1 and shows the distinct z-pattern. Morton codes are computationally efficient to calculate but as is also evident visually the curve may "jump" octants resulting in unfavorable communication patterns between nodes. The Hilbert curve on the other hand takes on a U shape, see Figure 2. Hilbert codes offer a "compact" mapping which may reduce the cost of communication but come at the cost in increased computational inefficiency.

As you can see in Figure 1, this is how you include an image in LaTeX.

As you can see in Figure 2, this is how you include an image in LaTeX.

3 Cornerstone Analysis

To evaluate the Cornerstone octree construction method under realistic conditions, we developed a configurable batch generation pipeline (`run_batches`) that sweeps over a matrix of particle distributions, particle counts, and perturbation magnitudes. Particles that leave the domain after perturbation are truncated back to the boundary.

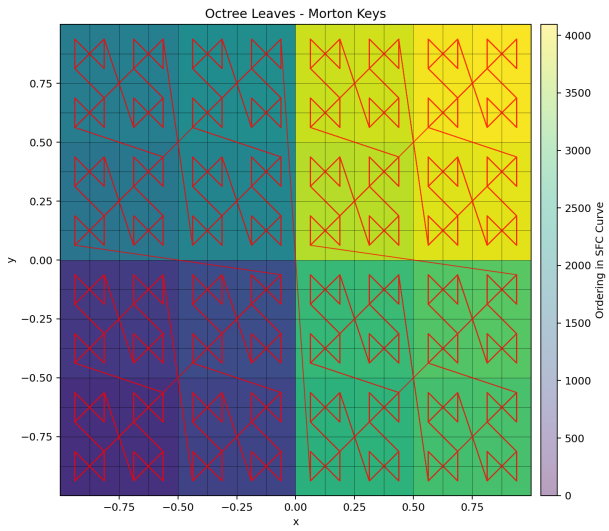

Figure 1. The leaves of an octree colored with respect to their Morton Z-order curve. The red trace plots the curves path through the leaves.

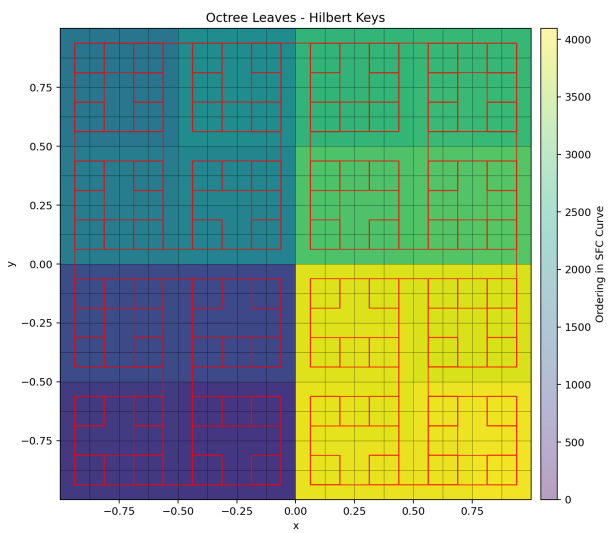

Figure 2. The leaves of an octree colored with respect to their Hilbert curve. The red trace plots the Hilbert curve path through the leaves.

3.1 Distribution Generators

Six initial particle distributions are used:

1. **`uniform_initial`**: Samples n particles uniformly from the cube $[-1, 1]^3$. A homogeneous baseline.

Table 1. Cornerstone Parameters

Parameter	Description
Precision	FP64 or FP32
θ	The MAC parameter
N_{crit}	Maximal # of points in a leaf
h	Smoothing parameter
L	The maximal depth of the octree
SFC	Morton or Hilbert

2. **`pancake_initial`**: A distribution where one axis is strongly compressed, resembling a disk or frisbee.
3. **`pancake_initial_tilted`**: Same as the pancake but rotated off-axis, so the thin dimension is no longer aligned with a coordinate plane.
4. **`filament_z_initial`**: A one-dimensional filament aligned along the z-axis.
5. **`filament_yz_initial`**: A filament lying in the yz-plane, anisotropic in two directions.
6. **`filament_xyz_initial`**: A filament oriented along the body diagonal, anisotropic in all three coordinate directions simultaneously.

3.2 Perturbation Model

After the initial distribution is generated, a perturbation vector is applied to simulate particle motion between timesteps. The perturbation function draws each displacement component independently from $\mathcal{U}(-s, +s)$ where s is the scale factor. Particles that move outside the domain after perturbation are truncated to the boundary (`out_of_bounds = 'truncate'`). Four perturbation scales are used:

- $s = 0.001$: Very small perturbation; particles barely move relative to the domain.
- $s = 0.01$: Moderate perturbation; noticeable displacement but locally coherent.
- $s = 0.1$: Large perturbation; significant displacement triggering substantial tree restructuring.
- $s = 0.5$: Very large perturbation; particles may traverse a large fraction of the domain.

3.3 Runtime Configuration

The Cornerstone framework accepts several runtime parameters via the command line:

- `-bucket-size-global`: Maximum number of particles per leaf node in the global octree (default: 1024).
- `-bucket-size-focus`: Maximum number of particles per leaf node in the focused octree (default: 64).
- `-theta`: The multipole acceptance criterion (MAC) parameter controlling the accuracy/performance tradeoff (default: 0.6).
- `-gpu`: Flag to enable GPU-accelerated octree construction.

No rotations were applied (`rotations = None`).

3.4 Test Configurations

The full sweep comprises 6 distributions × 5 particle counts × 4 perturbation scales = 120 configurations.

4 Project Status

4.1 Code

We have created the necessary Jupyter notebooks and python scripts to generate distributions of interest and their perturbations. This includes code to save those distributions to HDF5 files. We developed a framework in C++ for loading those HDF5 distributions and their perturbations and generating the corresponding octrees, adding the perturbations to the particles and then rebalancing the octree. This allows us to observe the effects of the initial construction as well as across varying magnitudes of perturbations. This will be important in developing our final report as it will be important to show how varying magnitudes of perturbations generate correlate to the rebalancing time for trees. This relates to Cornerstone's flagship feature which is allowing rebalancing rather than rebuilding.

4.2 Machines

Currently we have used CloudLab to gain access to GPUs allowing us to get preliminary results on a node with 4 V100s; however, access to this is difficult to attain on a regular basis, and hence we are hoping to obtain access to NCSA's Delta cluster in the coming weeks as this should allow better availability.

4.3 Algorithms

4.4 Results

5 Remaining Work

6 Related Work

The related work of your project [1].

7 Conclusion

This project is awesome.

8 Metadata

The code/data of the project can be found at:

https://github.com/Murdock022X/pca533-octrees

References

[1] Keller, S., Cavelan, A., Cabezon, R., Mayer, L., and Ciorba, F. Cornerstone: Octree construction algorithms for scalable particle simulations. In *Proceedings of the Platform for Advanced Scientific Computing Conference* (New York, NY, USA, 2023), PASC '23, Association for Computing Machinery.

Table 2. `uniform_initial` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			

Table 3. `pancake_initial` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			

Table 4. `pancake_initial_tilted` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			

Table 5. `filament_z_initial` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			

Table 6. `filament_yz_initial` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			

Table 7. `filament_xyz_initial` results.

N	Scale	Build (ms)	Rebalance (ms)	Depth
10^4	0.001			
10^4	0.01			
10^4	0.1			
10^4	0.5			
10^5	0.001			
10^5	0.01			
10^5	0.1			
10^5	0.5			
10^6	0.001			
10^6	0.01			
10^6	0.1			
10^6	0.5			
10^7	0.001			
10^7	0.01			
10^7	0.1			
10^7	0.5			
10^8	0.001			
10^8	0.01			
10^8	0.1			
10^8	0.5			